

How Blindspot works

This document is for anyone (a person or an AI coding agent) working on this repository, especially on the Bob IDE plugin in `plugin/`. Read it before changing anything. Section 9 lists the rules that must not be broken.

Status: 27 September 2026, after the final TEST runs. The experiment is finished and frozen (git tag `freeze`).

1. What Blindspot is, in one paragraph

Blindspot measures how well an AI coding agent (IBM Bob) understands a specific codebase. It generates an exam about the code directly from the code's syntax tree, so every answer is checked automatically. Bob sits the exam closed book, giving an answer and a confidence for each question. Blindspot marks the answers, finds the files and functions where Bob is wrong (especially confidently wrong), shows them in Bob IDE through the plugin, writes targeted context to fix them, and tests whether that context helped with a pre-registered experiment on a sealed test set.

The repository has two halves that only talk through files:

Half	Language	Folder	Owner	What it does
Engine	Python	<code>blindspot/</code> , <code>cli.py</code> , <code>tests/</code>	Nacky	Generates exams, runs Bob, marks, does statistics, writes result files
Plugin ("Bob Readiness")	TypeScript, VS Code extension	<code>plugin/</code>	Aziz	Reads those files and shows them inside Bob IDE; lets a developer send findings to Bob

The plugin never imports Python and never runs the statistics. The only command it runs from the engine is `python cli.py publish` (the Publish button).

2. Repository layout

<code>blindspot/</code>	Python engine (one module per job, listed in section 4)
<code>cli.py</code>	command line entry point: <code>python cli.py <command></code>
<code>tests/</code>	pytest suite for the engine (about 97 tests)
<code>PREREGISTRATION.md</code>	the experiment plan, committed before any results; Deviations section at the bottom
<code>docs/REPORT_SCHEMA.md</code>	contract for the exam report files (plugin reads these)
<code>docs/HOW_IT_WORKS.md</code>	this file
<code>exams/</code>	generated questions and answer keys (hidden from Bob by <code>.bobignore</code>)
<code>index.json</code>	scanner output: modules, classes, functions, params, defaults, raises, calls
<code>candidates.jsonl</code>	every generated question (2,268 for tinydb)
<code>train.jsonl</code>	120 TRAIN questions
<code>test.jsonl</code>	200 TEST questions, sealed by SHA-256 (never edit)
<code>split_report.json</code>	counts and the TEST hash
<code>pilot.jsonl</code>	30 pilot questions (never drawn from TEST)
<code>contexts/</code>	
<code>C1/AGENTS.md</code>	context for condition C1: written by Bob's own <code>/init</code> on tinydb
<code>C2/AGENTS.md</code>	context for condition C2: C1 plus the Cartographer's notes
<code>results/</code>	everything produced by runs (committed, hidden from Bob by <code>.bobignore</code>)
<code>answers/<run>.jsonl</code>	Bob's raw parsed answers per question
<code>raw/<run>/*.json</code>	Bob's full reply and stats for every call (audit trail)
<code>marked/<run>.jsonl</code>	answers joined with questions and marked correct or wrong
<code>report_<run>.json</code>	per-run report (see <code>docs/REPORT_SCHEMA.md</code>)
<code>report_latest.json</code>	copy of the newest report
<code>history.jsonl</code>	one summary line per report, for change over time
<code>costs.jsonl</code>	one line per Bob call with its Bobcoin cost
<code>cartographer/</code>	the Cartographer's notes per module and a <code>log.json</code>
<code>final_analysis.json</code>	the pre-registered analysis of TEST
<code>sim/</code>	SIMULATED versions of the report files (fake data, labelled "simulated": true)
<code>target/tinydb/</code>	the codebase under test: tinydb v4.9.0, pinned commit (ignored by git, fetched by a
<code>command)</code>	
<code>target.lock.json</code>	records exactly which commit was examined
<code>plugin/</code>	the VS Code extension (see <code>plugin/README.md</code> , <code>plugin/BUILD_PLAN.md</code>)

.cache/	exam workspaces and cached Bob replies (ignored by git)
.bob/	Bob configuration for this repo
.bobignore	hides exams/, results/, .cache/ and secrets from Bob (plugin/ should be listed too)

A run is named `<set>_<condition>_r<repeat>`, for example `test_C2_r1`.

3. The target codebase

- Repository: <https://github.com/msiemens/tinydb>, tag `v4.9.0`, commit `1e39ad3a2d9eca57efd9019fd089b5a04676da95` (MIT licence).
- Only the `tinydb/` package is examined. `tinydb/mypy_plugin.py` and `tiny` modules are excluded.
- Fetches into `target/tinydb` by `python cli.py target`. `python cli.py target --force` deletes and re-clones it clean. `python cli.py target --check` verifies the commit and that no agent context files (`AGENTS.md`, `.bob`, `CLAUDE.md`, ...) are present.
- `target/` is ignored by the outer git repo. `target/tinydb` is its own git clone.

All file paths in every result file are relative to the `tinydb` repo root, with forward slashes, for example `tinydb/table.py`. That is why the plugin should have `target/tinydb` open as its workspace (see section 7).

4. The pipeline, command by command

Run commands from the repository root with the virtual environment active. Anything that calls Bob must be run from a standalone PowerShell window (not the Bob IDE terminal) with `BOB_API_KEY` set; the runner refuses otherwise.

Order	Command	Module	Calls Bob?	Reads	Writes
1	<code>target</code>	<code>target.py</code>	no	GitHub	<code>target/tinydb</code> , <code>target.lock.json</code>
2	<code>gonogo</code>	<code>gonogo.py</code>	yes, 5 small calls		<code>results/gonogo.json</code>
3	<code>scan</code>	<code>scan.py</code>	no	<code>target/tinydb</code>	<code>exams/index.json</code>
4	<code>generate</code>	<code>generate.py</code> , <code>families/*</code>	no	<code>index.json</code>	<code>exams/candidates.jsonl</code>
5	<code>split</code>	<code>split.py</code>	no	<code>candidates</code>	<code>train.jsonl</code> , <code>test.jsonl</code> , <code>split_report.json</code>
6	<code>pilot</code>	<code>pilot.py</code>	no	<code>candidates</code> , <code>train</code>	<code>exams/pilot.jsonl</code>
7	<code>exam --condition C --set S</code>	<code>exam.py</code> = <code>runner.py</code> + <code>mark.py</code> + <code>report.py</code>	yes	<code>questions</code> , <code>contexts</code>	<code>answers</code> , <code>raw</code> , <code>marked</code> , <code>report</code> , <code>history</code> , <code>costs</code>
8	<code>cartographer</code>	<code>cartographer.py</code>	yes, 5 calls	<code>results/marked/train_C1_r1.jsonl</code>	<code>contexts/C2/AGENTS.md</code> , <code>results/cartographer/</code>
9	<code>git tag freeze</code>				locks the experiment; TEST cannot run without it
10	<code>analyze</code>	<code>analyze.py</code>	no	<code>results/marked/test_*.jsonl</code>	<code>results/final_analysis.json</code>
any	<code>simulate</code>	<code>simulate.py</code>	no	<code>exams/train.jsonl</code>	<code>results/sim/</code> (fake, labelled)
any	<code>report --set S --condition C</code>	<code>report.py</code>	no	<code>marked run</code>	<code>report files</code>

Order	Command	Module	Calls Bob?	Reads	Writes
any	<code>publish [--run NAME] [--sim] [--dest PATH]</code>	<code>publish.py</code>	no	a marked run	files inside the target repo (section 6)

4.1 Question generation (no AI involved)

`scan.py` parses every module with Python's `ast` and records modules, classes (with bases and methods), functions (params, literal defaults, exceptions raised in their own code, names they call) and imports. `@overload` stubs are skipped and the last definition of a name wins.

Five question families (`blindspot/families/`), every truth taken from the index:

Family	Example	Answer type
imports	Does <code>tinydb/queries.py</code> import from <code>tinydb.table</code> ?	yes/no
exists	Does class <code>Table</code> have a method named <code>update_all</code> ? (real names, names moved from another class, near misses, mutations)	yes/no
defaults	What is the default value of parameter <code>cond</code> of <code>Table.update</code> ?	Python literal
raises	Does <code>LRUCache.__getitem__</code> contain a <code>raise</code> for <code>KeyError</code> in its own code?	yes/no
calls	Does <code>Query.__ne__</code> contain a call to something named <code>_generate_test</code> ?	yes/no

Yes/no families are balanced 50/50. Ambiguous cases (re-exports, inherited built-in methods, one-hop raises) are skipped so that every truth is defensible. Each question has an `id`, `family`, `subkind`, `module`, `path`, `entity` (for example `tinydb.table:Table.update`, or `tinydb.table:<module>` for module-level questions), `type` (`tf` or `value`), `text` and `truth`.

4.2 The three conditions

Condition	Context Bob gets
C0	nothing
C1	<code>contexts/C1/AGENTS.md</code> , produced by Bob's own <code>/init</code> run in Bob IDE on a clean <code>target/tinydb</code>
C2	<code>contexts/C2/AGENTS.md</code> = the C1 file plus a section "Blindspot notes (facts checked against the code)" written by the Cartographer

Everything else (mode, flags, prompt, batch size, question order) is identical across conditions.

4.3 How an exam runs (`runner.py`)

1. A fresh workspace is built at `.cache/exam_ws/<condition>/`: a copy of `target/tinydb`, then every agent context file is deleted (`AGENTS.md`, `CLAUDE.md`, `.bob`, `.cursor`, ...), then `.bob/custom_modes.yaml` with the examinee mode is written, then only that condition's `AGENTS.md` is copied in. Nothing published into `target/tinydb` can reach an exam.
2. Questions are sorted by `id`, shuffled with seed 7 and cut into batches of 20. The batches are the same for every condition.
3. Each batch is one headless call: `bob run --format json --workspace <ws> --trust --accept-license --max-turns 2 --max-cost 1.0 --disable-tool-groups read,edit,command,browser,mcp --disable-mcp --disable-subagents --mode blindspot-examinee`, with the prompt on stdin. The go/no-go test proved this is closed book (a secret file could not be read) while `AGENTS.md` still reaches Bob.
4. Bob must return a JSON array of `{"id", "answer", "p"}` where `p` is its probability that its own answer is correct. Malformed items are dropped and count as missing.
5. If a call used any tool, all its answers are excluded (`reason: tool_use`). If more than 10% are missing, the batch is re-run once. Remaining gaps are excluded (`reason: missing`).

6. Replies are cached in `.cache/bob/` so a rerun never pays twice. Every call is logged in `results/costs.jsonl` and saved in `results/raw/<run>/`.
7. Running `--set test` is refused unless the git tag `freeze` exists and the TEST file still matches its SHA-256.

4.4 Marking (`mark.py`)

Yes/no answers must be JSON booleans equal to the truth. Value answers are compared as Python literals with `ast.literal_eval`, type included (0 is not `False`). Each marked row keeps `text`, `truth`, `answer`, `p`, `correct` (0 or 1), `excluded` and `reason`, so later steps can say exactly what Bob believed.

4.5 Reports (`report.py`, contract in `docs/REPORT_SCHEMA.md`)

For the run overall, each module (file) and each folder: accuracy with a Wilson 95% interval, Brier score, ECE, reliability bins, confidently wrong rate (wrong with $p \geq 0.8$), overconfidence (mean p minus accuracy), `low_n` (fewer than 10 answers) and `heat` = 1 minus the Wilson lower bound (0 fine, 1 worst). The red rule marks a module red if its Wilson lower bound is below 0.60 or its confidently wrong rate is above 0.20. `red_modules` is ordered worst first. `worst_entities` lists the 10 functions or classes with the most confidently wrong answers.

4.6 The Cartographer (`cartographer.py`)

Reads only the TRAIN run under C1 (never TEST). Targets the modules where Bob made at least one mistake on TRAIN, worst first, at most 5. For each, one Bob call in the custom mode `blindspot-cartographer`, which may read files but not edit or run anything, asks for at most 40 lines of verified facts about that module (classes and methods, parameters and defaults, what each function raises, surprising calls, plausible names that do not exist), aimed at the TRAIN mistakes without copying the questions. If a call fails, notes are generated from `exams/index.json` instead and logged as `index_fallback`. The notes are appended to the C1 text to make `contexts/C2/AGENTS.md`.

On the real run the targets were `storages`, `database`, `table`, `queries`, `utils`.

4.7 Analysis (`analyze.py`, `stats.py`)

Pairs the same TEST questions across C1 and C2 (a question counts only if scored in both). Primary: exact McNemar test on the questions where the two conditions disagree, two sided, alpha 0.05. Secondary with Holm correction: Brier difference (paired bootstrap), confidently wrong rate (McNemar), and C1 vs C0 if C0 was run. Sensitivity: cluster bootstrap of the accuracy difference by entity and by module.

5. Results (real, not simulated)

Run	Questions	Accuracy	Notes
pilot C0	30	90.0%	pipeline check, 0.012 Bobcoins
train C1	120	91.7%	10 wrong, drove the Cartographer, 0.056 Bobcoins
test C1	200	89.0% (83.9 to 92.6)	Brier 0.086, confidently wrong 3.0%
test C2	200	95.0% (91.0 to 97.3)	Brier 0.041, confidently wrong 3.0%

Primary result: C2 fixed 17 questions and broke 5. Exact McNemar $p = 0.017$, significant. Wrong answers fell from 22 to 10. Brier improved (Holm $p < 0.001$). Confidently wrong rate did not change. Cluster bootstrap by entity: +6.0 points (2.1 to 10.1).

Limitations to state honestly: one repository, one run per condition, a popular library Bob may have seen in training, and C2 contains more text than C1 with no length-matched control. Deviations are logged in `PREREGISTRATION.md`.

6. What `publish` writes (the plugin's second contract)

`python cli.py publish` reads one marked run (default: the newest file in `results/marked/`, which is `test_C2_r1`; choose with `--run test_C1_r1`; `--sim` uses `results/sim/`) and writes into `target/tinydb` (or `--dest`):

File	Purpose
<code>.bob/context/readiness.json</code>	The plugin's context file, format fixed by <code>plugin/BUILD_PLAN.md</code> section 2 and <code>plugin/src/contract.ts</code>
<code>.bob/rules/readiness-context.md</code>	Plain English list of facts Bob got wrong with the correct answers. Bob loads <code>.bob/rules</code> automatically
<code>AGENTS.md</code>	A block between <code><!-- blindspot:start --></code> and <code><!-- blindspot:end --></code> with readiness per file. Other text is kept; republishing replaces only the block
<code>.bob/blindspot/</code>	Copies of every <code>report_*.json</code> and <code>history.jsonl</code> , so the Exam tab works with <code>target/tinydb</code> open

How `readiness.json` is filled from the marked answers:

Field	Source
<code>repo</code>	name of the destination folder, pinned commit, time of publishing
<code>summary.readiness</code>	overall accuracy of the run
<code>summary.questions</code>	number of scored answers
<code>summary.sure_but_wrong</code>	number of wrong answers with <code>p >= 0.8</code>
<code>summary.bobcoins_spent</code>	sum of <code>results/costs.jsonl</code> (0 for simulated data)
<code>files[]</code>	one per module: <code>readiness</code> = module accuracy, <code>status</code> <code>ready</code> (<code>>= 0.85</code>), <code>review</code> (<code>>= 0.65</code>) or <code>not_ready</code> , <code>imports</code> from the scanner
<code>functions[]</code>	one per function, method or class that questions were about. <code>id</code> = <code><file>::<qualname></code> . Line range from the actual source (<code>ast</code>); for a class only its header lines. <code>readiness</code> = accuracy on its questions. <code>status</code> = <code>wrong</code> (any wrong answer with <code>p >= 0.8</code>), <code>part</code> (some wrong), <code>ok</code> (all right). <code>tested</code> = the short name appears somewhere under <code>tests/</code> (a name search, a heuristic). <code>misconceptions</code> = up to 5 wrong answers, most confident first: <code>believed</code> is the question plus Bob's answer, <code>truth</code> is the correct answer, <code>confidence</code> is <code>p</code>
<code>findings[]</code>	<code>review_risk</code> for every function with a wrong answer (severity high if confidently wrong at <code>p >= 0.9</code> , medium if confidently wrong, else low). <code>test_gap</code> when readiness is below 0.85 and no test mentions the name. <code>bob_allowed</code> uses the gate <code>yes</code> (<code>>= 0.85</code>), <code>with_notes</code> (<code>>= 0.65</code>), <code>no</code> (below). <code>estimated_coins</code> is a fixed 0.05. Line range is the first 5 lines of the function. Ids <code>F001</code> , <code>F002</code> , ... in severity order
<code>starter_tasks[]</code>	up to 3 small functions that Bob answered fully correctly, in files marked <code>ready</code>
<code>setup</code>	<code>pip install -e .</code> and <code>pytest -q</code>
<code>_blindspot</code>	extra key (simulated flag, red modules). The plugin's zod schema strips unknown keys, so it is ignored there

Choosing the run is a presentation choice: `test_C1_r1` shows Bob before Blindspot's notes (22 mistakes to highlight), `test_C2_r1` shows after (10 mistakes).

7. How the plugin connects

The plugin reads two things, both relative to the open workspace folder:

1. The exam reports (`docs/REPORT_SCHEMA.md`), looked for in this order: `bobReadiness.resultsPath` (default `results`), then `results/sim` (shown with a simulated banner), then `.bob/blindspot` (needs the small patch `plugin_report_fallback.patch` to `plugin/src/report/reportStore.ts`). Report paths are mapped onto `bobReadiness.targetRoot`, else `target/<repo>` from `target.lock.json`, else the workspace root.
2. The readiness context at `bobReadiness.contextPath` (default `.bob/context/readiness.json`), paths relative to the workspace root.

Two ways to open it:

Open this folder in Bob IDE	Works	Caveat
target/tinydb (recommended for the demo)	Highlights, hovers, CodeLens, Problems, all tabs, and the Exam tab via .bob/blindspot (with the patch)	Run publish first. The Publish button will not work here because cli.py is not in this folder
the repository root	Exam tab and tree colouring from results/, Publish button	Highlighting finds no context unless bobReadiness.contextPath is set to target/tinydb/.bob/context/readiness.json, and its paths would then not match files; use the tinydb folder for highlighting

The Publish button runs `bobReadiness.publishCommand` (default `python cli.py publish`, using `.venv` if present) in the workspace root.

The "Send to Bob" flow creates a branch and needs a clean working tree. `publish` modifies `target/tinydb`, so commit inside that folder first (`cd target/tinydb; git add -A; git commit -m "Add Blindspot context"`). Do not push that commit. This flow has only been tested with the plugin's fake Bob so far.

8. Common tasks

Demo setup on a fresh machine:

```
git pull
python cli.py target          # fetch tinydb if missing
python cli.py publish --run test_C1_r1
# open target/tinydb in Bob IDE
```

Show the improvement in the video: `publish test_C1_r1`, record, then `python cli.py publish --run test_C2_r1`, record again. The Exam tab's Compare runs can show `test_C1_r1` against `test_C2_r1` at any time.

Work on the plugin without real data: `python cli.py simulate` then `python cli.py publish --sim`. Afterwards run `python cli.py target --force` to wipe simulated files out of `target/tinydb`.

Run the engine tests: `pytest -q` (expect about 97 passed, 1 skipped). Plugin: `cd plugin; npm test; npm run build`.

9. Rules that must not be broken

1. Never edit `exams/test.jsonl`, `exams/train.jsonl`, `exams/split_report.json`, `contexts/` or anything in `results/` except by the commands that produce them. The TEST file is sealed by hash.
2. Do not change the parameters in `blindspot/config.py` that are listed in `PREREGISTRATION.md` section 8; `tests/test_prereg.py` fails if they drift.
3. Field names in `docs/REPORT_SCHEMA.md` and in the readiness contract are fixed. Changes go through Nacky and must be made on both sides at once.
4. Never run `/init` or any Bob task on `target/tinydb` while it contains published or simulated files; reset with `python cli.py target --force` first.
5. Simulated data must always carry `"simulated": true` and be labelled in the UI.
6. Bob exams run only from a standalone terminal, never the Bob IDE terminal.
7. Never commit `.env` or the Bob API key. One branch per change, pull request, merge; no direct commits to `main`.
8. Line endings are LF (`.gitattributes`). On Windows this matters for the TEST hash.